



클린 코드를 위한 행동 패턴 & MVC 패턴

Do it! 클린 프로그래밍 · 7장 7-4 ~ 7-5

전략 · 옵저버 · 반복자 · 상태 · 중재자 패턴과 MVC 패턴까지 책의 예제를 함께 짚어 봅니다.



OVERVIEW

오늘 함께 볼 여섯 가지 패턴

1

전략 패턴

행위를 캡슐화해 런타임에 교체

2

옵저버 패턴

상태 변화를 일대다로 알림

3

반복자 패턴

내부 구조 숨기고 순차 접근

4

상태 패턴

상태별 동작을 클래스로 분리

5

중재자 패턴

N:M 관계를 M:1로 단순화

6

MVC 패턴

관심사의 분리 (7-5)



행동 패턴은 무엇을 다루나

핵심 정의

객체 간의 상호 작용 방식과 책임 분배 방법을 제시하는 디자인 패턴이다.

- 복잡한 흐름 제어를 단순화
- 객체 간 결합도를 최소화
- 효율적인 상호 작용을 가능하게 함

이 절에서 다루는 5가지

- 전략 패턴
- 옵저버 패턴
- 반복자 패턴
- 상태 패턴
- 중재자 패턴



전략 패턴 · 행위를 갈아끼운다

기존 코드를 수정하지 않고 새로운 전략을 추가하거나 런타임에 교체할 수 있게 한다.

책 예제 · 배달 앱 결제 수단

- Payment 인터페이스 — pay(amount) 정의
- CreditCard · SamsungPay · KakaoPay 구현
- DeliveryApp(Context) 가 전략을 보유
- setPayment() 로 결제 수단을 실시간 교체

적용된 객체 지향 원칙

개방 - 폐쇄	새 결제 수단은 클래스 추가만으로 확장
의존성 역전	고수준 모듈이 추상 인터페이스에 의존
다형성	같은 인터페이스로 여러 결제 호출
캡슐화	구체 구현은 숨기고 pay만 노출



옵저버 패턴 · 바뀌면 알려준다

관찰 대상(Subject)의 상태가 바뀌면, 관찰자(Observer) 모두에게 **일대다(one-to-many)** 로 알림을 전달한다.

책 예제 · 유튜브 구독 알림

- Subscriber 인터페이스 — notify() 선언
- YoutubeChannel(Subject) 이 구독자 리스트 보유
- register / remove / notify Subscriber
- 콘텐츠 업로드 → 등록된 모두에게 알림

왜 좋은가 — 느슨한 결합

- Subject는 Observer가 어떻게 구현됐는지 알 필요가 없다
- 새 관찰자를 추가해도 내부 코드를 수정하지 않는다
- 발행 / 구독(pub-sub) 모델로도 불린다
- 시스템 확장과 유지 보수가 편리해진다



반복자 패턴 · 내부를 몰라도 순회한다

이터레이터를 사용해 컬렉션 요소에 **순차 접근** — 내부 구조를 노출하지 않는다.

책 예제 · 회원 목록 순회

- Iterator — hasNext() / next() 정의
- NameIterator · DateIterator 로 기준별 순회
- MemberInfo 가 nameIterator / dateIterator 제공
- 같은 데이터를 이름순 · 가입일순으로 출력

주의할 점

- 자바는 Iterator · Enumeration을 기본 제공한다
- 굳이 직접 만들면 클래스 수 · 복잡도가 늘 수 있다
- 기본 반복문보다 성능이 다소 불리할 수 있다

→ 단일 책임 원칙은 지키되, 무조건 적용은 지양



상태 패턴 · 상태마다 다르게 행동한다

객체의 상태에 따라 서로 다른 동작을 수행 — 복잡한 **분기 로직**을 **단순화**한다.

책 예제 · Caps Lock 키보드

- CapsLockState — input / inputAndChangeState
- On · Off 가 상태별 동작을 구현
- On은 대문자 변환(아스키 +32), Off는 그대로
- KeyBoard(Context) 가 현재 상태를 보유

실행 결과

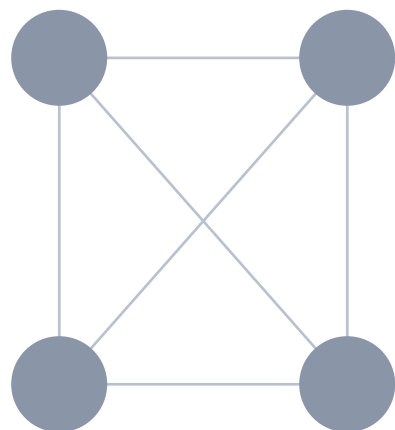
a → b → A → b → B

- 상태마다 클래스 분리 → 단일 책임 원칙
- 새 상태 추가 시 기존 코드 수정 불필요 → 개방-폐쇄

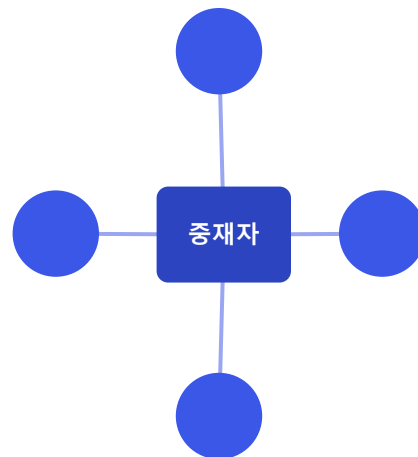
중재자 패턴 · 다대다를 가운데로 모은다

복잡한 N:M 객체 관계에 중재자를 도입해 **M:1**로 단순화한다.

적용 전 (N:M)



적용 후 (M:1)



책 예제 · 그룹 채팅

- ChatSystem(Mediator) · User(Colleague)
- ChatRoom 이 참여자를 보유하고 메시지 중계
- 발신자만 빼고 나머지 모두에게 전달

옵저버 = 단방향 / 중재자 = 양방향

단, 중재자에 책임이 몰리면 제거가 어렵고 비대해질 수 있음



행동 패턴 한눈에 정리

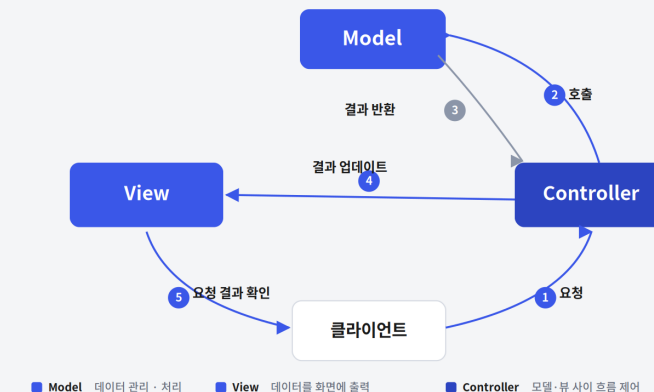
패턴	한 줄 핵심	책 예제
전략	행위를 캡슐화해 런타임 교체	배달 앱 결제 수단
옵저버	상태 변화를 일대다로 알림 (단방향)	유튜브 구독 알림
반복자	내부 구조 숨기고 순차 접근	회원 목록 순회
상태	상태별 동작을 클래스로 분리	Caps Lock 키보드
중재자	N:M → M:1 단순화 (양방향)	그룹 채팅



7-5 MVC PATTERN

MVC 패턴 · 관심사의 분리

역할을 Model · View · Controller 셋으로 나눠, 각자 **자신의 역할에만 집중**하게 한다.



Model

데이터를 가진 객체

- 요청 결과 데이터 반환
- 뷰·컨트롤러 정보 미포함
- 변경 알림 기능 구현

View

사용자 인터페이스 담당

- 모델 데이터를 화면 출력
- 데이터 임의 저장 금지
- 변경 알림 기능 구현

Controller

모델·뷰 사이 흐름 제어

- 모델·뷰를 모두 알고 있음
- 입력 받아 모델에 반영
- 변경 사항을 조율·중재



동작 흐름과 구현 방식

MVC 동작 흐름 (웹 사이트)

- 1 요청 — 사용자 요청을 컨트롤러가 수신
- 2 호출 — 컨트롤러가 모델을 호출
- 3 결과 반환 — 모델이 처리 후 결과 반환
- 4 결과 업데이트 — 컨트롤러가 뷰에 전달
- 5 결과 확인 — 사용자가 화면에서 확인

모델 1

JSP가 뷰 + 컨트롤러를 동시에. 빠르지만 로직·UI 혼재

모델 2

컨트롤러를 서블릿이 담당. 분리도 ↑, 대규모에 유리

Spring MVC

DispatcherServlet · HandlerMapping · ViewResolver 활용



7-5 EXAMPLE

책 예제 · 회원 정보 출력 기능

UserModel

데이터만 보유

id · name · grade
게터/세터 제공

UserView

화면 출력만 담당

displayUserInfo()
로 회원 정보 출력

UserController

모델·뷰를 중재

둘을 멤버로 보유
setUserName·updateView

컨트롤러가 모델과 뷰를 멤버로 들고 중재 — 최소 기능만으로도 관심사의 분리가 그대로 드러난다



정리하자면

모든 패턴은 결국 결합도를 낮추고 변경에 강한 구조를 위한 도구

행동 패턴

객체 간 상호작용과 책임 분배를 정리 — 결합도 ↓

옵저버 vs 중재자

단방향 알림 vs 양방향 중계, 헛갈리지 말 것

MVC

관심사의 분리 — Model·View·Controller 각자의 역할

가장 중요한 원칙

패턴은 적합할 때만 효과적, 무조건 적용은 지양